Reading the Foveal Window — a team plan

Oct 24, 2026 · @Luiz Velho: A student team will test, in about eight weeks, whether the search that finds the eyes' vergence can also classify the foveal window — and whether those classes make the stereo matcher more robust at depth edges and half-occlusions. The first milestone is a cheap oracle experiment that decides whether the rest is worth building.

Why this project

The remaining errors of the foveated stereo engine sit at depth edges and half-occlusions, and four earlier fixes did not move them. This project tests a new angle on that problem.

Where things stand. The engine (repository `visgraf/fov-3d-vision`) renders foveated stereo pairs in Blender and runs an active gaze loop on top. Phase D ended with an honest account:

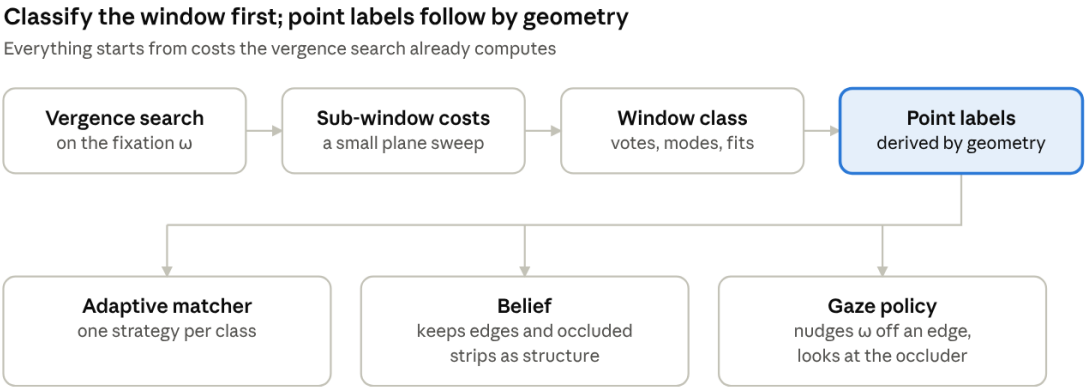
- About a tenth of the judged area is truly wrong; about 8% of the area sits at depth edges and half-occlusions. That is the current block matcher's floor at one look.
- Coarse-to-fine search and a smaller window did not help. No single-cell confidence feature was a usable test (best AUC 0.71).
- Context did help: a neighbourhood test was the best signal, and two looks that disagree contain a bad one 94% of the time.

The conjecture (Luiz, 24 Oct 2026). The principles behind the nominal-vergence search can classify the foveal window itself. They can separate local properties of the fixated point ("near a half-occlusion") from global properties of the window ("contains a depth edge"), so the matcher can switch to adaptive strategies and be more robust.

Why a team. The work splits into five parts with clear interfaces. Its first milestone is a go/no-go, so the team must decide together, from evidence, whether to continue.

The idea at a glance

The window is classified first; each point's label is then derived from the window's structure by geometry, not detected on its own.



From the vergence search to three consumers · 7 steps

- **Global.** Keep the cyclopean matching cost per sub-window instead of summing it, and let each sub-window vote for its best disparity. One mode means one surface (a plane fit gives its slant); two modes mean two layers, a boundary and a depth jump; a comb plus self-similarity means repeated texture; little gradient along ϕ means low texture; anything else is clutter.
- **Local.** Given two layers, the half-occluded strip is predicted: on the far layer next to the boundary, seen only by the eye on the far layer's side, about as wide as the disparity jump, and absent for boundaries along the epipolar direction.
- **Scale.** A 2° fovea spans about 7 cm at 2 m and 17 cm at 5 m, so one or two layers is the common case.

Questions and decision rules

Three questions, asked in order; each has a rule written before its run, so the data cannot be argued into agreement afterwards. The thresholds below are proposals: the team ratifies or changes them at kick-off, and after that only by a logged decision made before the run.

#	Question	Experiment	Decision rule
H1	Would perfect labels help the matcher?	E0 — labels from Blender truth drive the adaptive matcher, against the current matcher on the same pairs	Go on only if edge and occlusion outliers fall by a third or more (about 8% of judged area to 5.3% or less), with fine-band coverage down no more than 5%
H2	Can the window be classified from the images alone?	E1 — the vote-histogram classifier, scored against truth	One-surface vs two-layer accuracy of 80% or more; occluded-strip recall and precision of 70% or more
H3	Do estimated labels keep the gain?	E2 — classifier labels drive the matcher on the same pairs as E0	Estimated labels keep at least half of E0's reduction

If H1 fails, the project stops there and writes up why: that negative result is a full deliverable, not a failure of the team.

Work packages

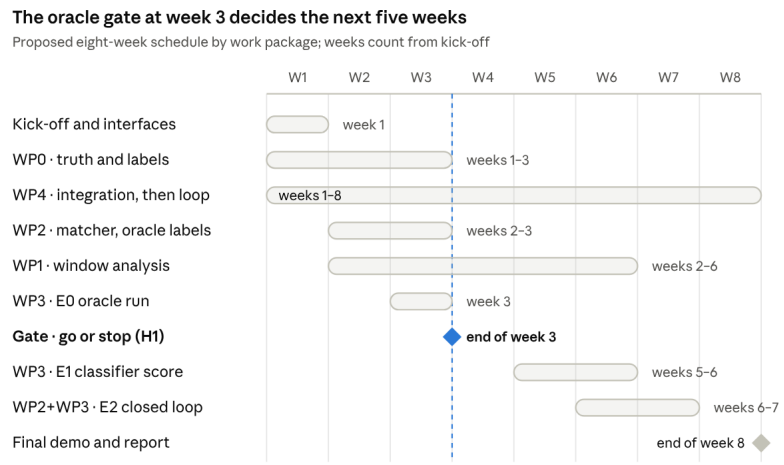
Five work packages, one owner each; the plan assumes a team of five and merges WP3 into WP4 for a team of four. Owners are chosen at kick-off — write your name in the last column.

Package	Delivers	Needs	Feeds	Owner
WP0 · Truth and labels	A set of foveal pairs with per-point truth: layer, boundary band, half-occluded strip and which eye sees it	The renderer's depth and position passes; the saved 500-fixation record, or new renders of the classroom	WP1, WP2, WP3	
WP1 · Window analysis	Per-sub-window cost volume from the cyclopean search; vote histogram, modes, plane and boundary fits, period and texture tests; window class and point labels with probabilities	Pairs from WP0	WP2 (in E1–E2), WP3	
WP2 · Adaptive matcher	One strategy per class on top of the current matcher, switched by labels; a baseline mode identical to today's	Labels from WP0 (E0) or WP1 (E2)	WP3	
WP3 · Evaluation	The judge and its metrics (outliers at edges and occlusions, fine coverage, depth error); runs E0–E2; checks every decision rule; the figures	Outputs of WP1 and WP2; truth from WP0	The decision log, the report	
WP4 · Integration	Before the gate: branch, run scripts, quick checks, the decision log. After it: labels into the belief and the gaze policy	Everyone's code	The loop, the final demo	

Open question for week 1 (WP0): does the saved record keep the foveal pairs and position passes, or must pairs be rendered again?

Milestones

The oracle result at the end of week 3 decides whether weeks 4–8 build the classifier or write up a negative result. The schedule below is a proposal for an eight-week course; stretch or compress it at kick-off, but keep the gate before the classifier.



proposed schedule · 8 weeks, 1 gate

If the gate says stop, weeks 4–5 go to the write-up: what E0 showed, why the labels are not the bottleneck, and what that implies for the matcher.

Interfaces agreed in week one

Five shared formats are fixed in week 1, so the packages can work in parallel from week 2. After that, an interface changes only by a logged decision taken with every owner present.

Interface	Producer	Consumers	Holds
Pair record	WP0	all	Left and right foveal windows sampled in epipolar coordinates (θ, ϕ), with the fixation ω, its vergence and the level
Label map	WP0 (truth), WP1 (estimate)	WP2, WP3	Per sample: layer, boundary band, occluded strip and its eye, low texture, periodic — each with a probability. Per window: class, modes, plane fit, boundary line, depth jump
Matcher call	WP2	WP3, WP4	Pair and optional labels in; disparity, confidence and occlusion mask out; a baseline switch that reproduces today's matcher exactly
Metrics table	WP3	all	One row per look and label class, fixed columns, written by one script
Test windows	WP1	all	Five synthetic windows with known answers — frontal, slanted, two layers, periodic, textureless — that every package runs before it pushes

Team roles and rituals

Each person owns one package and rotates through two team roles; disagreements are settled by evidence or by a cheap experiment, and dissent is recorded, not erased.

Role	Who	Does
Package owner	One student per WP	Delivers the package, keeps its interface, demos it weekly
Reviewer	Rotates weekly	Reviews every merge from outside the package; asks “how do you know?”
Scribe	Rotates weekly	Keeps the decision log and the week’s numbers
Integrator	WP4 owner	Keeps the branch runnable; merges only what passes the test windows
Client	Luiz	States the goals, ratifies the rules, attends the gate; not a day-to-day manager

Rituals

1. **Kick-off (week 1).** Choose owners, ratify the decision rules, fix the five interfaces.
2. **Weekly demo (30 min).** Each owner shows one picture or one number, and says what changed. No slides.
3. **Decision log.** Numbered entries, as in the project (D24, D25…): date, question, evidence, decision, who agreed, who dissented. A rule is logged before its run.
4. **Run budget.** Interactive under 10 s, batch under 5 min; anything longer runs overnight, only by agreement.
5. **Pairing at the seams.** Owners of packages that share an interface pair for one session a week (WP0 with WP1 on labels, WP1 with WP2 on the matcher call).
6. **Gate meeting (end of week 3).** WP3 reads E0 against H1’s rule; the team decides go or stop together.
7. **Retrospectives.** At the gate and at the end: what helped the team work, what got in the way, what to change.

Working with AI agents in the team

Agents are welcome as drafters, explainers and coders; people own every decision. The project already works this way: a planning conversation writes the plan and the prompt, a coding agent runs it on the workstation, and a person reads the results.

1. **Agents draft, people decide.** Every entry in the decision log is signed by a person.
2. **Prompts are shared.** A prompt that produced code, a number or a claim goes in the repository with a one-line summary of the answer, so others can rerun and criticize it.
3. **Claims are checked before they enter the log.** A number is reproduced by a script, a formula by a test window or a derivation, a citation by opening it.
4. **Ask for detail.** Asking an agent to explain its own result in more depth is a cheap check: in this project it caught a formula off by a factor of 4.
5. **Give it the context.** Point the agent at this plan, the interfaces and the log; a short prompt works when the shared context is long.
6. **Pose conjectures, not orders.** Ask for the refinement, the caveats and a test, as in the lecture's example prompt.
7. **One voice per package.** The owner steers the agent on that package; changes that cross packages go through the integrator.
8. **Keep a record of misses.** Note when an agent was wrong and how it was caught; bring it to the retrospective.

Risks and fallbacks

The largest technical risk is circularity; the largest team risk is one package blocking the others. Each risk has a signal to watch and a fallback agreed in advance.

Risk	Signal	Fallback
Classes built from the same costs that fail at edges	E1 misses H2 near boundaries	Labels as probabilities; a clutter class that falls back to the baseline; add monocular edge cues
Oracle helps, classifier cannot follow	E2 keeps less than half of E0's gain	Use labels only where confident; ask the loop for a second, different look at uncertain windows
Thin structures and three-layer windows (chairs, cables)	Many windows classed as clutter	Baseline matcher there; report the clutter share
Boundaries along the epipolar direction	Residual errors gather on those boundaries	A monocular edge map as an extra cue
The saved record lacks pairs or truth	WP0's week-1 finding	Render a new pair set of the classroom overnight, within a budget agreed at kick-off
The project switches to a learned matcher	A logged decision to switch	WP2 re-targets: labels set the network's range, choose when to call it and gate its output; E0 reruns on the new baseline
One package blocks the others	Two missed weekly demos	The integrator cuts scope and pairs owners at that seam; the gate date does not move

For discussion in class

Bring a short answer to each; we will use them to run a mock kick-off.

1. Which of the agent's claims in this plan would you check first, and how?
2. Where did the human steer the exchange, and where did the agent? What would a vaguer prompt have produced?
3. Is H1's go/no-go rule well posed? What would make you change it after seeing data — and is that allowed?
4. How would your team split the five packages, and which interface must be agreed first?
5. What would you cut if the course had four weeks instead of eight?
6. What should the team write down so that a new member — or an AI agent — can pick up the work tomorrow?

Sources and related material

From this project

- Lecture deck *Vergence from Visual Data* — the geometry, matching cost, servo, precision and failure modes this plan builds on.
- Case-study deck *Thinking with an AI Agent* — the exchange that produced this plan.
- Repository visgraf/fov-3d-vision: the Phase D summary and the decision log D1–D23.
- L. Velho, Experiments on Active 3D Vision, VISGRAF TR-09-2026.
- L. Velho, Personal Notes on a Methodology for R&D in Applied Computational Mathematics, VISGRAF TR-00-2026.

Background reading for WP1 and WP2 — listed from the agent's knowledge, not looked up; check each before citing it (rule 3 above).

- T. Kanade, M. Okutomi, "A stereo matching algorithm with an adaptive window," IEEE TPAMI, 1994.
- K.-J. Yoon, I. S. Kweon, "Adaptive support-weight approach for correspondence search," IEEE TPAMI, 2006.
- M. Bleyer, C. Rhemann, C. Rother, "PatchMatch Stereo — stereo matching with slanted support windows," BMVC, 2011.
- S. S. Intille, A. F. Bobick, "Disparity-space images and large occlusion stereo," ECCV, 1994.
- D. Geiger, B. Ladendorf, A. Yuille, "Occlusions and binocular stereo," IJCV, 1995.
- D. J. Fleet, A. D. Jepson, M. Jenkin, "Phase-based disparity measurement," CVGIP: Image Understanding, 1991.